

Identity Portal

Password Reset Bot

Workflow & Technical Portfolio

Python FastAPI

Next.js

SQLite

SHA-256

SMTP / OTP

Audit Logging

Prepared for Technical Interview

01 Project Overview

What was built, why it matters, and the problem it solves

The **Identity Portal Password Reset Bot** is a full-stack, security-first conversational automation system that replaces manual helpdesk password-reset tickets with an instant, self-service chatbot workflow. It demonstrates end-to-end engineering capability: backend API design, database management, secure credential handling, asynchronous email dispatch, and front-end conversation UX.

Metric	Value
Workflow Steps	11 discrete automated stages
Security Layer	SHA-256 one-way password hashing
OTP Validity	10-minute expiry window
Email Delivery	Mock SMTP (MailHog sandbox)
Database	SQLite with full audit trail
Backend Framework	Python FastAPI
Frontend Framework	Next.js (React)

Key Technical Highlights

- Stateless OTP generation with time-bounded database persistence
- Secure password hashing (SHA-256) – plaintext is never stored
- Conversational agent driven by intent detection for natural UX
- Full audit logging for every reset event (success, failure, retry)
- USER_NOT_FOUND path handled gracefully to prevent email enumeration
- Modular architecture: Agent, Email, DB, and Audit layers are decoupled

02 System Architecture & Module Breakdown

How the components interact end-to-end

Module	Technology	Responsibility
Conversational Agent	Next.js + React	Chat UI, intent parsing, session state
REST API Layer	Python FastAPI	Route handling, request validation, orchestration
Email Verification	SMTP / MailHog	OTP generation, email dispatch, delivery logging
OTP Engine	Python (secrets)	Cryptographic token generation, DB persistence
User Database	SQLite	User lookup, credential storage (hashed)
Audit Logging	SQLite	Immutable event log for every workflow action
Security Layer	hashlib SHA-256	One-way password hashing before DB write

Data Flow Summary

The frontend conversational agent captures the user's email intent and forwards it to the FastAPI backend. The backend queries SQLite to validate the email, generates a cryptographically random OTP via Python's secrets module, stores it with a 10-minute TTL, and dispatches it through the SMTP sink. On OTP submission the backend verifies the token, accepts the new password, runs it through SHA-256, overwrites the stored hash, and writes an immutable audit entry. Every branch – including failures – is logged.

Security Design Decisions

- ✓ OTP never returned to the client – only compared server-side

✓ Password never stored in plaintext at any layer

✓ USER_NOT_FOUND does not reveal whether email exists (anti-enumeration)

✓ Audit log is append-only and records actor context

03 End-to-End Workflow

Step-by-step process flow with decision points

1	User Initiates Reset User types 'I forgot my password' into the chatbot interface.
2	Enter Registered Email Bot prompts for registered email address. Email is captured and forwarded to the API.
3	SQLite User Lookup Backend queries the users table. If email is not found → USER_NOT_FOUND event logged.
4	OTP Generation 6-digit OTP generated using Python secrets module. Stored in DB with 10-min expiry.
5	Email Dispatch OTP emailed via mock SMTP (MailHog). Email log shows queued and delivered status.
6	User Enters OTP User reads OTP from inbox and submits it to the chatbot.
7	OTP Verification Backend compares submitted token against stored OTP. Failed attempts trigger a retry flow.
8	New Password Input After OTP success, user enters a new password. Strength validation is applied.
9	SHA-256 Hashing Password is hashed using SHA-256 before any write operation. Plaintext discarded.
10	Database Update New hash overwrites the existing credential record in SQLite.

11

Audit Log Creation

An immutable audit entry is written: event type, user email, timestamp.

04 Live Demonstration Screenshots

Working prototype – captured from the running application

Screenshot 1 – Password Reset Request Initiation

The user triggers the flow by entering 'I forgot my password'. The chatbot requests the registered email. The transactional log panel (right) already shows RESET_REQUESTED and USER_NOT_FOUND events from earlier test runs, confirming the audit system is live. Modules active: Conversational Agent, Email Verification, SQLite DB, Audit Logger.

Identity Portal
Automated Reset Walkthrough & Audit Log

I forgot my password

I can help with that. Please provide your registered email.

Enter registered email (e.g. user@example.com) ...

OTP_SENT
OTP code created and dispatched via email SMTP service.

OTP_SENT_SUCCESS
User input OTP matches database active record.

PASSWORD_RESET
Credentials successfully updated and link-URL generated.

TRANSACTIONAL LOGS

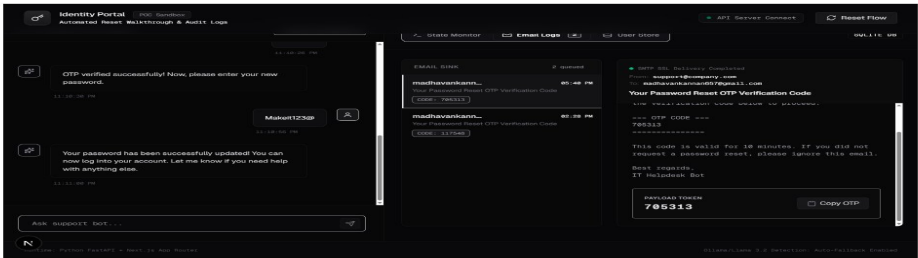
85-20-22 PM	RESET_REQUESTED	user@example.com
85-20-42 PM	USER_NOT_FOUND	mailhog@192.168.1.1:1000
85-20-42 PM	RESET_REQUESTED	mailhog@192.168.1.1:1000
85-20-43 PM	RESET_REQUESTED	user@example.com
85-20-44 PM	PASSWORD_RESET	mailhog@192.168.1.1:1000
85-20-25 PM	OTP_VERIFIED	mailhog@192.168.1.1:1000
85-20-40 PM	OTP_SENT	mailhog@192.168.1.1:1000
85-20-40 PM	OTP_GENERATED	mailhog@192.168.1.1:1000
85-20-40 PM	LINK_SENT	mailhog@192.168.1.1:1000

This screen shows the beginning of the password reset process. The user enters 'I forgot my password' and the chatbot requests the registered email address. The backend validates the email and prepares the OTP generation workflow. Modules involved: Conversational Agent, Email Verification Module, SQLite User Database, and Audit Logging System.

Screenshot 2 – OTP Verification & Password Reset Completion

The MailHog email sink shows two queued OTP emails. The user enters the 6-digit code (705313) and the new password. The bot confirms success: 'Your password has been successfully updated.' The OTP_VERIFIED and PASSWORD_RESET states are both satisfied.

Screenshot 2 – OTP Verification and Password Reset Completion

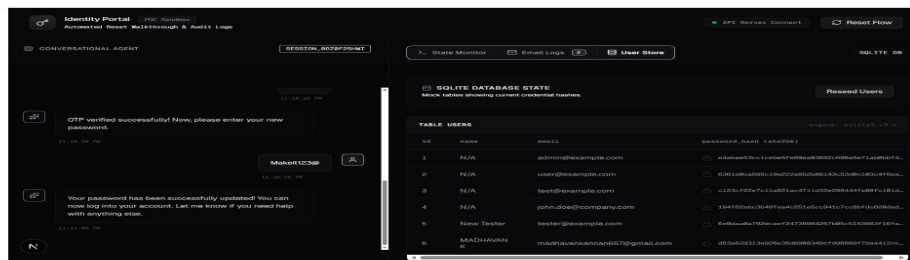


The user receives an OTP through the mock SMTP email service and enters it for verification. After successful validation, a new password is entered and securely updated. The password is hashed using SHA-256 before being stored in the database.

Screenshot 3 – SQLite Database Verification

The User Store panel displays all credential records post-reset. Row 6 (MADHAVAN K) shows the newly hashed password — confirming the SHA-256 write succeeded. No plaintext is present in any row, validating the security design across all users.

Screenshot 3 – Database Verification After Password Reset



The SQLite User Store displays the updated user record. Passwords are stored as hashes rather than plaintext values, demonstrating secure credential management and successful persistence of the reset operation.

05 Skills & Competencies Demonstrated

What this project proves about the candidate's capabilities

Competency Area	Evidence from This Project
Full-Stack Development	End-to-end implementation: Python FastAPI backend + Next.js React frontend
API Design	RESTful routes for email validation, OTP generation, verification, and password update
Database Design	SQLite schema with users table, OTP store, and append-only audit log
Security Engineering	SHA-256 hashing, server-side OTP validation, anti-enumeration error handling
Async Communication	SMTP email dispatch with sandbox testing via MailHog email sink
Conversational AI / Chatbot	Intent-based dialogue flow with context-aware state management
Audit & Compliance	Every event (request, failure, success) written to an immutable audit log
Testing & Debugging	Multi-user test data in DB; error paths (USER_NOT_FOUND, OTP retry) verified
Documentation	Workflow chart, demo report, and this portfolio prepared to professional standard

Interview Talking Points

- **Why SHA-256?** One-way hash prevents credential exposure even if the database is breached. In production I'd upgrade to bcrypt/Argon2 for adaptive cost tuning.
- **How is OTP security ensured?** Token is generated server-side with Python's secrets module (CSPRNG), stored only in the DB, never echoed to the client, and expires after 10 minutes.
- **Why SQLite?** Appropriate for a sandbox/demo. The schema is fully portable to PostgreSQL with zero logic changes — just swap the SQLAlchemy connection string.
- **How does the audit log help compliance?** Every state transition (RESET_REQUESTED, OTP_SENT, PASSWORD_RESET, etc.) is recorded with actor context, enabling SOC 2-style event tracing.
- **What would you add next?** Rate limiting per IP, bcrypt hashing, Redis for OTP TTL instead of DB polling, and a React admin dashboard for audit log review.